Universidad de las Fuerzas Armadas-ESPE

Nombre: Gutierrez Pérez Dayanara Belén.

NRC: 36466.

Fecha: 23/10/2026.

Carrera: Ing. Telecomunicaciones.

Departamento Electrónica, Eléctrica y Telecomunicaciones.

Ejercicios de programación y algoritmia

1. Media de dos valores

Desarrollar un programa que ingrese dos números por teclado e imprima su media.

1.1.Análisis del problema

Entradas: Dos números ingresados desde el teclado (n1 y n2).

Proceso: Suma de ambos números y el resultado dividir para 2 que es el número total de datos.

Salida: Muestra el valor de la media.

Restricción: Solo ingresar números.

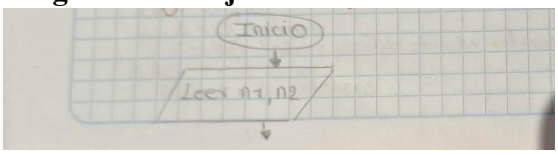
1.2.Tabla de objetos

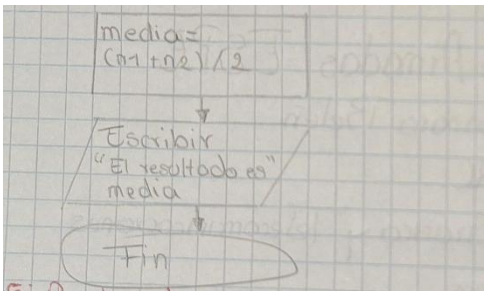
Objeto	Nombre	Valor	Tipo
Dato de entrada Primer numero	N1	Var	Entero
Dato de entrada Segundo numero	N2	Var	Entero
Resultado Media	Media	Var	Entero
Número de datos Divisor	2	Var	Entero

1.3.Seudocódigo

Inicio
Leer n1, n2
Media=(n1+n2) /2
Escribir “el resultado es”, media
Fin

1.4.Diagrama de flujo

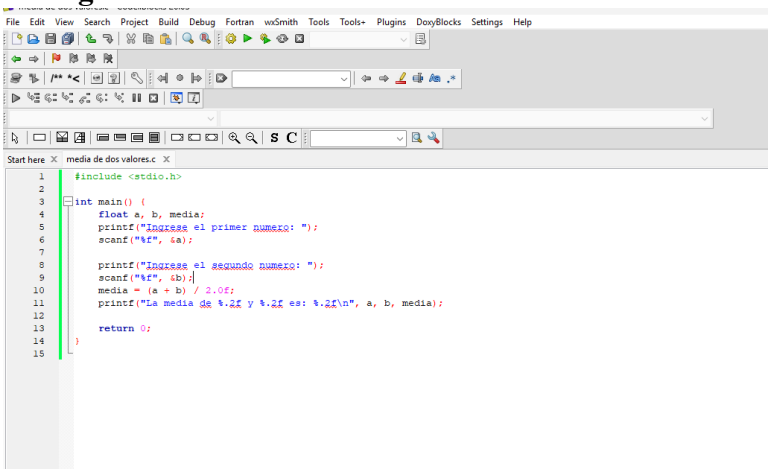




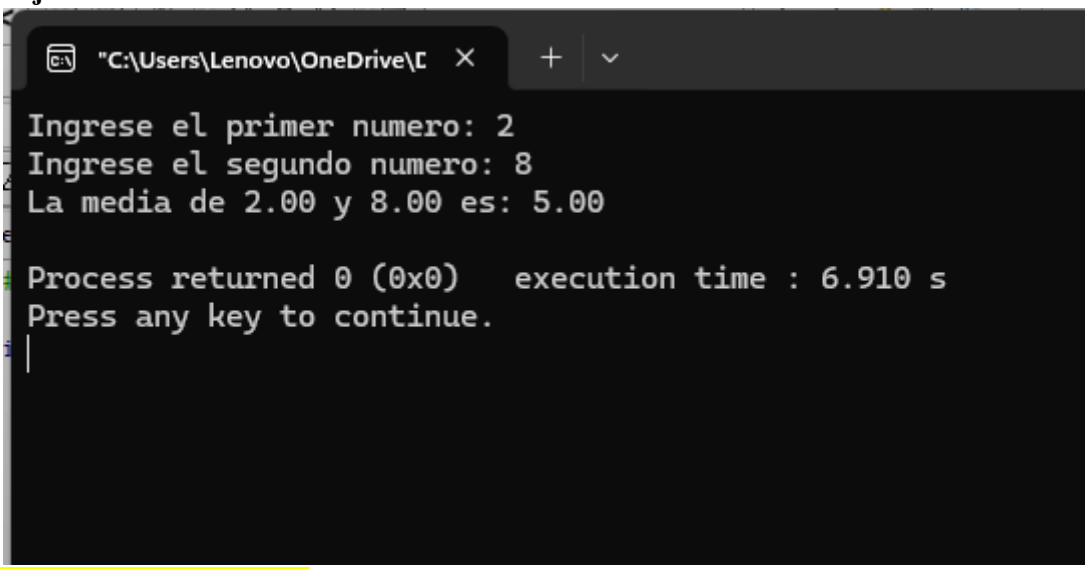
1.5.Prueba de escritorio

Caso	N1	N2	(n1+n2) /2	Salida
Normal (entero)	4	4	(4+4) / 2	El resultado es 4
Negativo y positivo	-6	8	(-6+8) /2	El resultado es 1
Norma (entero)	6	9	(6+9) / 2	El resultado es 7.5

1.6. Código



1.7. Ejecución



2. Valor absoluto al cubo

Desarrollar un programa que lea un numero por teclado y escriba por pantalla de |x|^3.

1.1.Análisis del problema

Entradas: Un número real x ingresado.

Proceso: Calcular el valor absoluto de |x| y elevarlo al cubo.

Salida: Muestra el valor absoluto de |x|^3.

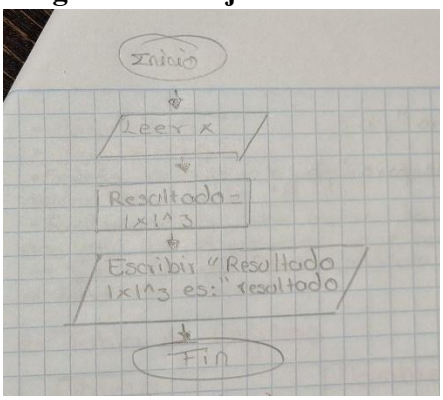
1.2.Tabla de objetos

Objeto	Nombre	Valor	Tipo
Dato de entrada (numero)	x	Var	real
Resultado	Resultado	Var	Real

1.3.Seudocódigo

Inicio
Leer x
Resultado= |x|^3
Escribir “resultado |x|^3 es:”, resultado
Fin

1.4.Diagrama de flujo



1.5.Prueba de escritorio

Caso	X	|x|	|x|^3	Salida
Normal (positivo)	2	2	8	Resultado |x|^3= 8
Normal (negativo)	-3	3	27	Resultado |x|^3= 27
Limite (0)	0	0	0	Resultado |x|^3= 0

1.6. Código

```c
#include <stdio.h>
#include <math.h>

int main() {
    double x, cubo, valor_absoluto;

    printf("Ingrese el valor de x: ");
    scanf("%lf", &x);

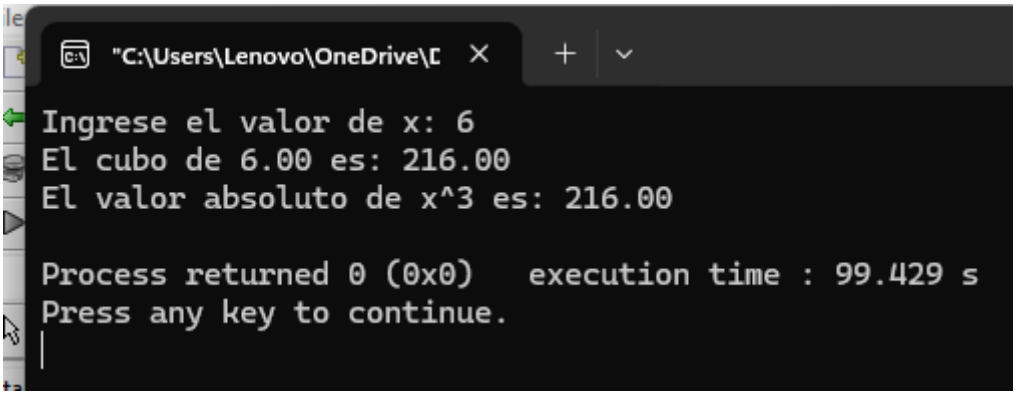
    cubo = x * x * x;

    valor_absoluto = fabs(cubo);

    printf("El cubo de %.2lf es: %.2lf\n", x, cubo);
    printf("El valor absoluto de x^3 es: %.2lf\n", valor_absoluto);

    return 0;
}
```

1.7. Ejecución



3. Divisible

Desarrolla un programa que le a dos números enteros por teclado y determine si el primero es divisible por el segundo. Se mostrará por pantalla el resultado.

3.1.Análisis del problema

Entradas: dos números enteros, dividendo (a) y divisor(b)

Procesos: a es divisible para b y si su resultado es igual a cero (a % b =0)

Salida: muestra si a es divisible para b o no

Restricciones: El divisor (b) no puede ser 0 ya que la división para 0 no esta definida.

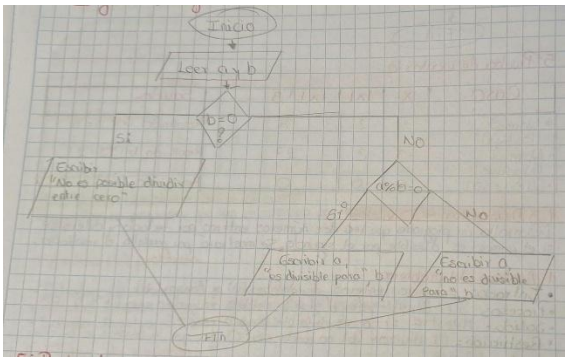
3.2.Tabla de objetos

Objeto	Nombre	Valor	Tipo
Dato de entrada (dividendo)	a	Var	entero
Dato de entrada (divisor)	b	Var	entero
Cero	0	Cte	entero

3.3.Seudocódigo

Inicio
Leer a y b
Si b=0 entonces
Escribir “no es posible dividir entre cero”
Sino
Si a % b=0 entonces
Escribir a, “es divisible para”, b
Sino
Escribir a, “no es divisible para”, b
Fin
Fin
Fin

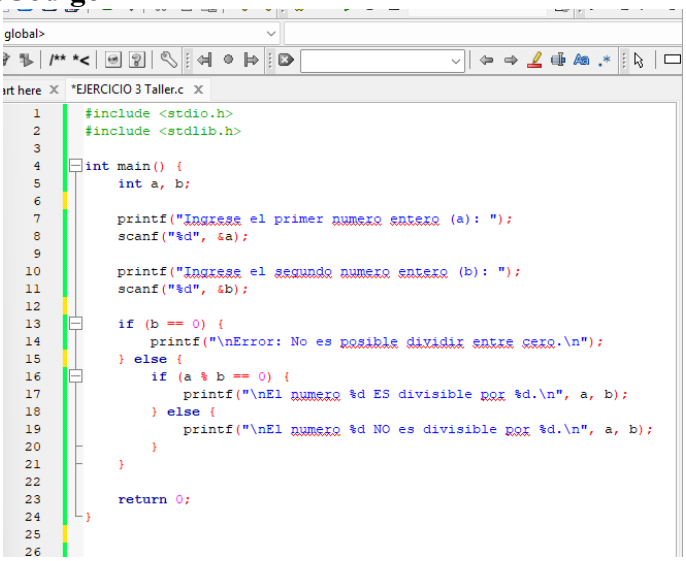
3.4.Diagrama de flujo



3.5. Prueba de escritorio

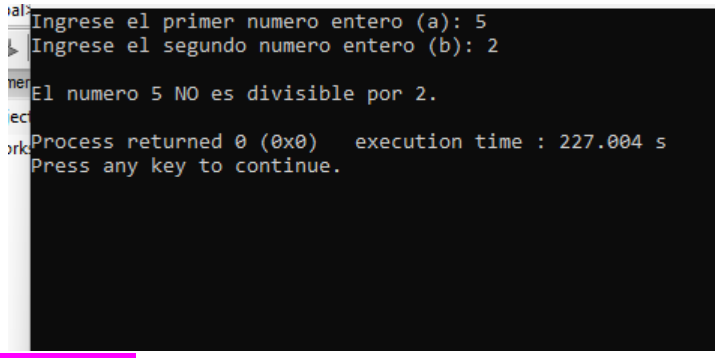
Caso	a	b	A % b =0	B=0	Salida
Normal (%)	20	5	0	no	20 es divisible para 5
Normal (no %)	7	2	1	no	7 no es divisible para 2
Mismo valor	2	2	0	no	2 es divisible para 2
Dividendo 0	0	17	0	no	0 es divisible para 17
Error	4	0	1	si	No es posible dividir entre 0

3.6.Código



```
global>
rt here  *EJERCICIO 3 Taller.c
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  int main() {
5      int a, b;
6
7      printf("Ingrese el primer numero entero (a): ");
8      scanf("%d", &a);
9
10     printf("Ingrese el segundo numero entero (b): ");
11     scanf("%d", &b);
12
13     if (b == 0) {
14         printf("\nError: No es posible dividir entre cero.\n");
15     } else {
16         if (a % b == 0) {
17             printf("\nEl numero %d ES divisible por %d.\n", a, b);
18         } else {
19             printf("\nEl numero %d NO es divisible por %d.\n", a, b);
20         }
21     }
22
23     return 0;
24 }
25
26
```

3.7.Ejecución



4. Intervalo

Desarrolla un programa que lea un número por teclado y determine si pertenece al intervalo [0,10] indicando el resultado por pantalla.

4.1. Análisis del problema

Entrada: un numero x ingresado

Proceso: determinar si x pertenece al intervalo [0, 10], comprobado por dos condiciones.

Sea x >=0

Sea x <=0

Salida: mensaje en la pantalla indicando que el numero ingresado esta dentro del intervalo [0, 10].

Restricciones: se debe separar la expresión lógica (0 <=x <=10) en dos para que el programa logre comprender lo que se esta pidiendo. Ya que, si lo presentamos como una sola expresión, va a trabajar como si fuera una sola orden.

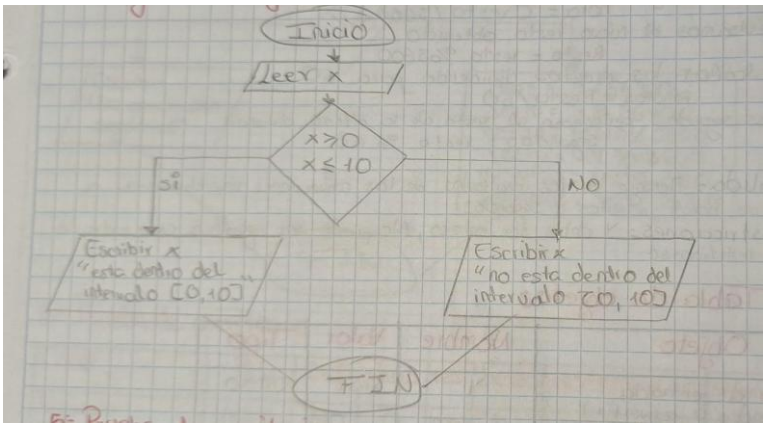
4.2. Tablas de objeto

Objeto	Nombre	Valor	Tipo
Dato de entrada (numero)	x	Var	Real
Limite inicial	0	Cte	Entero
Limite inicial	10	Cte	Entero

4.3. Seudocódigo

Inicio
Leer x
Si x => 0 y x<=10 entonces
Escribir x, “está dentro del intervalo [0,10]”
Sino
Escribir x, “no está dentro del intervalo [0,10]”
Fin
Fin

4.4. Diagrama de flujo



4.5.Prueba de escritorio

Caso	**x**	**X=>0**	**X<=0**	**X=>0 X<=10**	**Salida**
Dentro del intervalo	8	Verdadero	Verdadero	Verdadero	8 esta dentro del intervalo [0,10]
Limite inicial	0	Verdadero	Verdadero	Verdadero	0 está dentro del intervalo [0,10]
Limite final	10	Verdadero	Verdadero	Verdadero	10 está dentro del intervalo [0,10]
Menor que el mínimo	-1.8	Falso	Verdadero	Falso	-1.8 está dentro del intervalo [0,10]
Mayor que el max	13	Verdadero	Falso	Falso	13 está dentro del intervalo [0,10]

4.6.Código

media de dos valores.c | valor absoluto.c | intervalo.c

```c
#include <stdio.h>

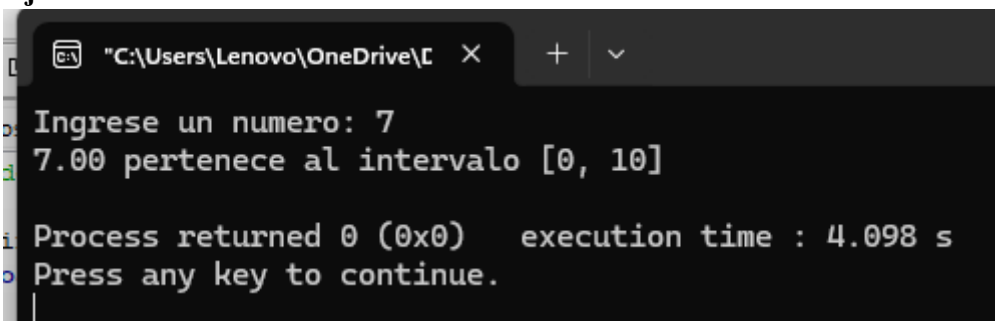
int main() {
    float x;

    printf("Ingrese un numero: ");
    scanf("%f", &x);

    if (x >= 0 && x <= 10) {
        printf("%.2f pertenece al intervalo [0, 10]\n", x);
    } else {
        printf("%.2f no pertenece al intervalo [0, 10]\n", x);
    }

    return 0;
}
```

4.7.Ejecución



5. Conversión de unidades de tiempo

Desarrolla un programa que lea por teclado un valor entero c. interpretándolo como un numero de segundos que dura un evento, y calcule y muestre por pantalla cuantos días, horas, minutos y segundos representan.

5.1.Análisis del problema

Entradas: un valor entero Y que representa los segundos a convertir

Proceso: calculamos el total de segundos y lo dividimos para la cantidad de segundos que tiene un día completo 24h = 86,400s

Días = [Y/86, 400]

- Para saber cuántos segundos quedan después de tomar los enteros, utilizamos el residuo de la operación anterior. Resto = Y % 86, 400
- Para calcular las horas dividimos el resto antes obtenido para el numero de segundos de una hora (3600)
- Revisamos el nuevo resto obtenido. Resto = resto % 3600
- Calcular los minutos dividiendo para 60. Minutos = resto / 60
- Los segundos pertenecen al resto de la última división. Segundos = resto % 60

Salida: mensaje de los resultados de las conversiones respectivamente a los días, horas, minutos y segundos.

Restricciones: y debe ser entero, no puede ser negativo caso contrario debe notificarse.

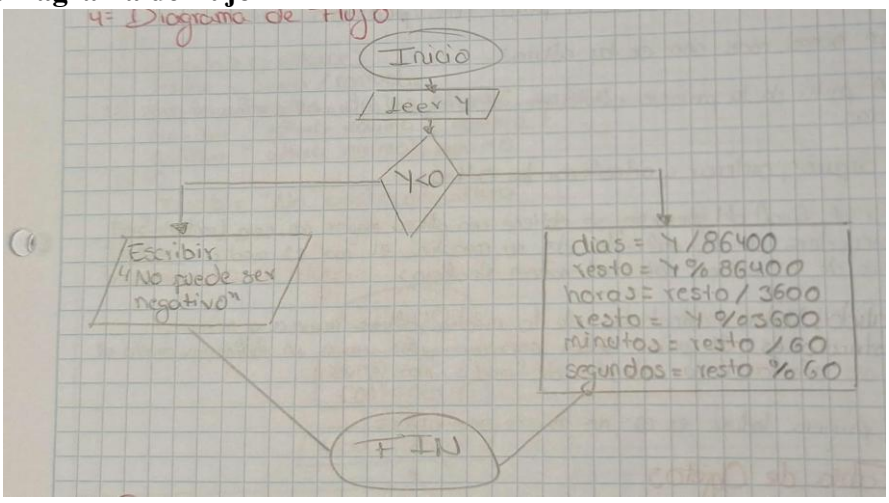
5.2.Tabla de objetos

Objeto	Nombre	Valor	Tipo
datos de entrada (segundos a convertir)	Y	Var	Entero
Días	Días	Var	Entero
Horas	Horas	Var	Entero
Minutos	Minutos	Var	Entero
Segundos calculados restantes	Segundos	Var	Entero
Residuo	Residuo	Var	Entero
Segundos días	86400	Cte	Entero
Segundos hora	3600	Cte	Entero
Segundos minutos	60	Cte	Entero

5.3.Seudocódigo

Inicio
Leer Y
Si Y<0 entonces
Escribir “no puede ser negativo”
Sino
Días = Y / 86400
Resto = Y % 86400
Horas = resto / 3600
Resto = Y% 3600
Minutos= resto / 60
Segundos = resto % 60
Escribir Y “segundos es igual a”, días, “días”, horas, “horas”, minutos, “minutos” y “segundos”
Fin
Fin

5.4.Diagrama de flujo



5.5.Prueba de escritorio

Caso	Y	Y <0	Días	Rest	Horas	Rest	Minutos	Segundos	Salida
	10000	no	0	10000	2	2800	46	40	0 dias, 2 horas, 46 minutos y 40 segundos
Mas de un día	90000	No	1	3600	1	0	0	0	1 dias, 1 horas, 0 minutos y 0 segundos
Minutos	65	No	0	65	0	65	1	5	0 dias, 0 horas, 1 minutos y 5 segundos
Valor 0	0	No	0	0	0	0	0	0	0 dias, 0 horas, 0 minutos y 0 segundos
Valor negativo	-50	Si	-	-	-	-	-	-	No puede ser negativo

5.6.Código

```c
#include <stdio.h>

int main() {
    int c, dias, horas, minutos, segundos, resto;

    printf("Ingrese la cantidad de segundos (c): ");
    scanf("%d", &c);

    if (c < 0) {
        printf("El numero de segundos debe ser >= 0\n");
    } else {
        dias = c / 86400;
        resto = c % 86400;

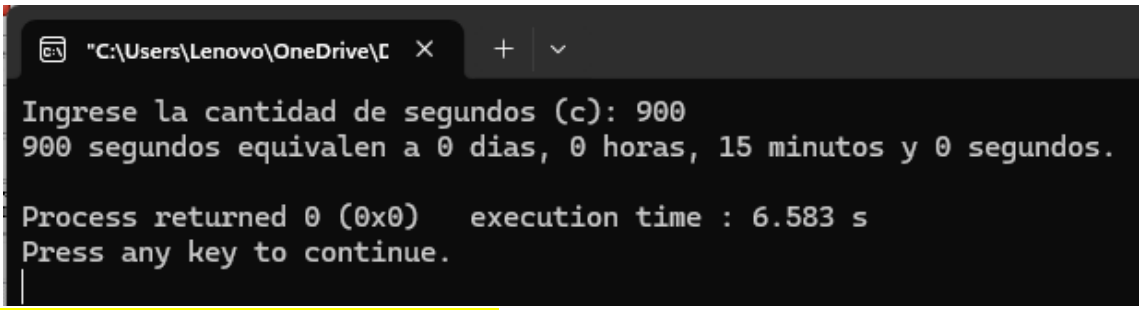
        horas = resto / 3600;
        resto = resto % 3600;

        minutos = resto / 60;
        segundos = resto % 60;

        printf("%d segundos equivalen a %d dias, %d horas, %d minutos y %d segundos.\n",
               c, dias, horas, minutos, segundos);
    }

    return 0;
}
```

5.7.Ejecución



6. Máximo, mínimo y media de alturas

Desarrollar un programa que lea por teclado un número indeterminado de alturas y deje de leer cuando se introduzca una negativa. El programa debe calcular e imprimir la media, el máximo y el mínimo de las alturas introducidas.

6.1. Análisis del problema

Entradas: numero indeterminado que representara las alturas. La lectura finaliza cuando se ingresa cualquier número negativo.

Proceso: utiliza un bucle que nos ayudara para las alturas siempre y cuando no haya negativos.

- se necesita tener un contador para tener precisión
- se suma cada una de las alturas
- a partir de la primera altura se establecerán máximos y mínimos para las alturas
- si al final del proceso, se obtienen una altura mayor que cero (contador >0) se proceder con el calculo de su media el cual es media = suma / contador la suma de las alturas, sobre el número de las alturas.

Salida: valores calculados de la media, altura máxima y mínima.

Restricciones: el primer valor negativo sirve como un indicador de alto el cual no afecta su cálculo de media, Max y min

6.2. Tabla de objetos

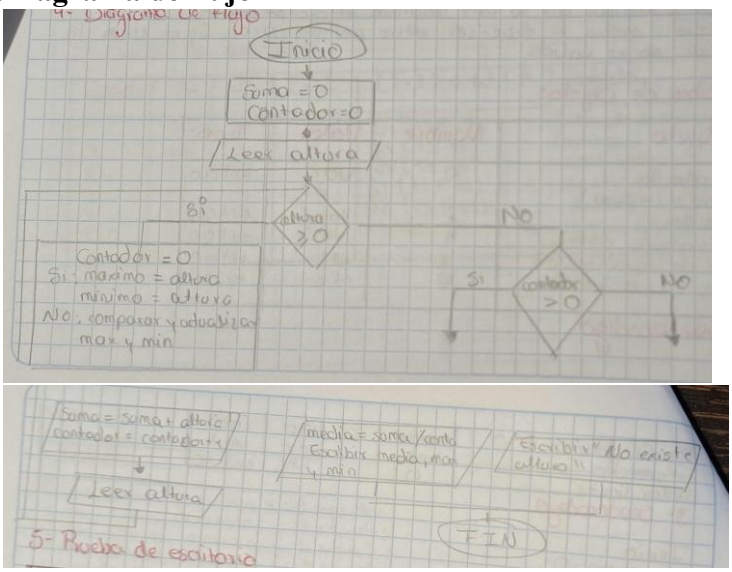
Objetos	Nombre	Valor	Tipo
Dato de entrada (altura leída)	altura	Var	Real
Acumulador de la suma	Suma	Var	Real
Contador de la altura valida	Contador	Var	Entero
Mayor altura	Máximo	Var	Real
Mínima altura	Mínimo	Var	Real
Promedio	Media	Media	Real
Valor limite	0	Cte	Entero

6.3. Seudocódigo

Inicio
Suma=0
Contador =0
Leer altura
Altura=>0 hacer
Si contador=0 entonces

Máximo= altura
Mínimo=altura
Sino
Si altura > máximo entonces
Máximo= altura
Fin
Si contador > 0 entonces
Media = suma / contador
Escribir “media de las alturas”, media
Escribir “altura máxima”, máximo
Escribir “altura mínima”, mínimo
Sino
Escribir “no existe una altura”
Fin
Fin

6.4.Diagrama de flujo



6.5.Prueba de escritorio

Caso	altura	Altura>0	contador	Suma	Max	min	Salida
1	1.70	Si	1	1.70	1.70	1.70	Inicia max y min en 1.70
2	1.85	Si	2	3.55	1.85	1.70	Actualizar max 1.85
3	1.65	Si	3	5.20	1.85	1.65	Actualizar min 1.65
4	-1.00	No	3	5.20	1.85	1.65	Sale bucle
fin			3	5.20	1.85	1.60	Media= 5.20 / 3= 1.73

6.6. Código

```c
#include <stdio.h>

int main() {
    float altura, suma = 0.0, maximo = 0.0, minimo = 0.0, media;
    int contador = 0;

    printf("Ingrese una altura (valor negativo para terminar): ");
    scanf("%f", &altura);

    while (altura >= 0) {
        if (contador == 0) {
            maximo = altura;
            minimo = altura;
        } else {
            if (altura > maximo) maximo = altura;
            if (altura < minimo) minimo = altura;
        }

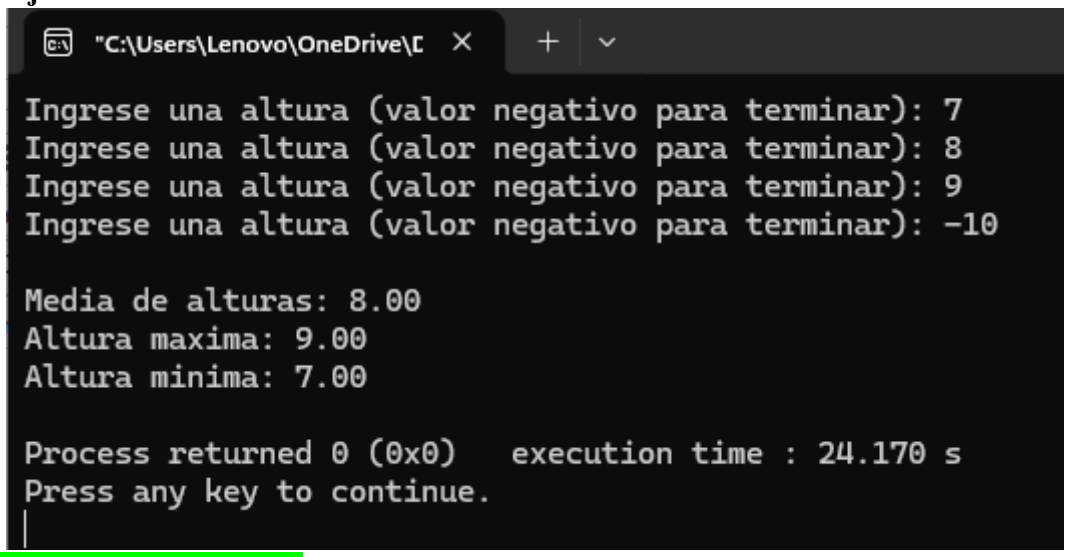
        suma += altura;
        contador++;

        printf("Ingrese una altura (valor negativo para terminar): ");
        scanf("%f", &altura);
    }

    if (contador > 0) {
        media = suma / contador;
        printf("\nMedia de alturas: %.2f\n", media);
        printf("Altura maxima: %.2f\n", maximo);
        printf("Altura minima: %.2f\n", minimo);
    } else {
        printf("\nNo se ingresaron alturas validas.\n");
    }

    return 0;
}
```

6.7.Ejecución



7. Raíz cuadrada entera

Desarrollar un programa que calcule la raíz cuadrada entera por defecto de un numero N positivo dado que se define como el mayor numero entero R>0 tal que R^2 <= N.

7.1.Análisis del problema

Entrada: un numero entero positio N

Proceso: iniciamos con i= 0

Con un bucle incrementa el valor de i de uno en uno, mientras su cuadrado es menor o igual (i^2 <=N)

Cuando i^2 <N, obteneos la raíz cuadrada entera restando -1. R = i-1

Salida: indicar que la raíz cuadrada entera por defecto

Restricción: numero N debe ser entero positivo (N>0). Si es (N<=0) la entrada no es válida.

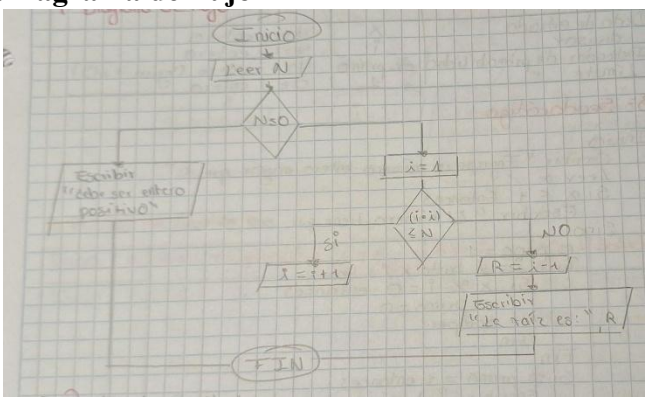
7.2. Tabla de objetos

Objeto	Nombre	Valor	Tipo
Dato entrada (número a evaluar)	N	Var	Entero
Contador	i	Var	Entero
Raíz cuadrada entero	R	Var	Entero
Valor de inicialización	1	Cte	Entero
Límite de restricción	0	Cte	Entero

7.3. Seudocódigo

Inicio
Leer N
Si N<= 0 entonces
Escribir “debe ser entero positivo (N>0)”
Sino
I=1
Mientras (i*i) <=N hacer
I=i+1
Fin
R=i-1
Escribir “la raíz cuadrada por defecto de”, N, “es”, R
Fin
Fin

7.4. Diagrama de flujo



7.5. Prueba de escritorio

Caso	N	i	I*i	(i*i) <=N	R	Salida
	16	1	1	Si (1<=16)		Entra al bucle

1	16	2	4	Si (1<=16)		Continua
2	16	3	9	Si (1<=16)		Continua
3	16	4	16	Si (1<=16)		Continua
4	16	5	25	No (1<=16)		Salida
Fin	16	5	25	No	4	La raíz de 16 es 4

7.6.Código

media de dos valores.c | valor absoluto.c | intervalo.c | convercion de unidades de tiempo.c | maximo, minimo y

```c
#include <stdio.h>

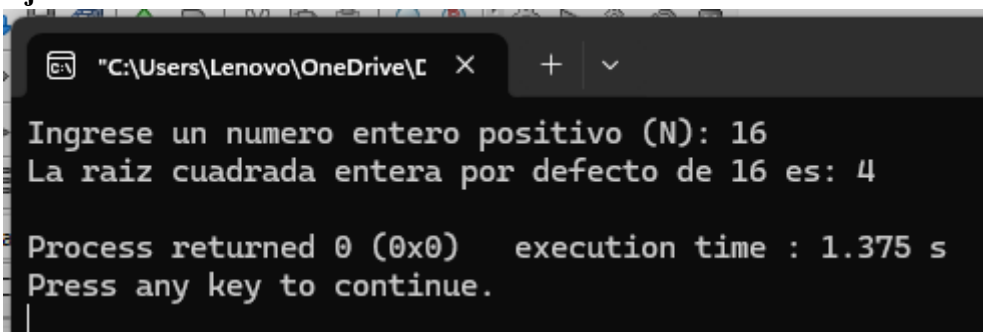
int main() {
    int N, i = 1, R;

    printf("Ingrese un numero entero positivo (N): ");
    scanf("%d", &N);

    if (N <= 0) {
        printf("El numero debe ser un entero positivo (N > 0)\n");
    } else {
        while ((i * i) <= N) {
            i++;
        }
        R = i - 1;
        printf("La raiz cuadrada entera por defecto de %d es: %d\n", N, R);
    }

    return 0;
}
```

7.7.Ejecución



8. Número primo

Desarrolla un programa que ingrese por teclado a un numero entero mayor que 1 y determine si es primo o no.

8.1.Análisis del problema

Entrada: número entero x

Proceso: un numero primo (x) es un numero que solo se divide de forma exacta entre 1 y el mismo

Salida: un mensaje indicando el numero ingresado es primo o no

Restricción: x debe ser mayor a 1

8.2.Tabla de objetos

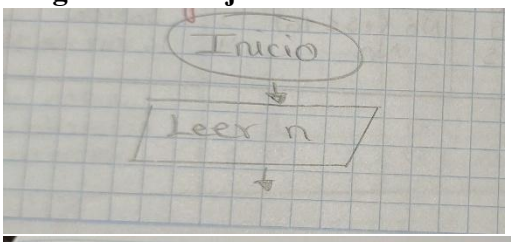
Objeto	Nombre	Valor	Tipo

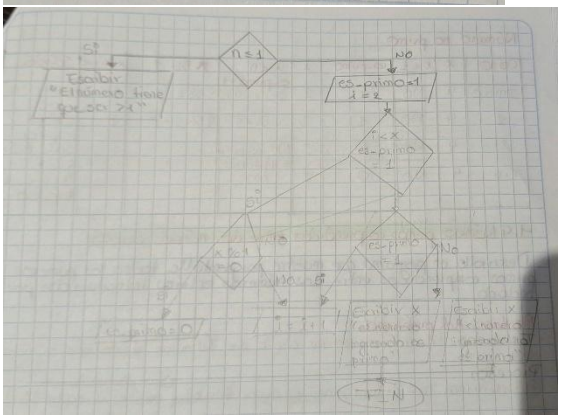
Dato de entrada	X	Var	Entero
Divisor	i	Var	Entero
Indicador de primalidad	Es_primo	Var	Entero (lógica 1 o 0)
Limite	1	Cte	Entero

8.3. Seudocódigo

Inicio
Escribir “ingrese un numero entero mayor que 1:”
Leer x
Si x <=1 entonces
Escribir “el numero tiene que ser >1”
Sino
Es_primo =1
Para i= 2 hasta x / 2 hacer
Si x % i=0 entonces
Es_primo=0
Romper
Fin
Fin
Si es_primo =1 entonces
Escribir x, “el número ingresado es primo”
Sino
Escribir x, “el numero ingresado no es primo”
Fin
Fin
Fin

8.4.Diagrama de flujo





8.5.Prueba de escritorio

Caso	**x**	**N<=1**	**Es-primo**	**i**	**I<x es_primo=1**	**X % i**	**X % i=0**	**salida**
Inicio	7	No	1	2	Si (2<7)	7 % 2=1	No	Incrementa i a 3
1	7	No	1	3	Si (3<7)	7 % 3=1	No	Incrementa i a 4
2	7	No	1	4	Si (4<7)	7 % 4=1	No	Incrementa i a 5
3	7	No	1	5	Si (5<7)	7 % 5=1	No	Incrementa i a 6
4	7	No	1	6	Si (6<7)	7 % 6=1	No	Incrementa i a 7
5	7	No	1	7	No (7<7)			Sale del bucle
Fin	7	No	1	7	No			Salida 7 numero primo

Entrada no valida

1	1	si						El numero debe ser un entero mayor a 1

Numero no primo

Inicio	9	No	1	2	Si (2<9)	9 % 2 =1	No	Incrementa i a 3
1	9	No	1	3	Si (3<9)	9 % 3=0	Si	Cambia es_primo a 0
2	9	No	0	3	No (es_primo=1) falso			Sale del bucle
Fin	9	No	0	3	No			Numero primo

8.6.Código

```c
#include <stdio.h>

int main() {
    int n, i = 2, es_primo = 1;

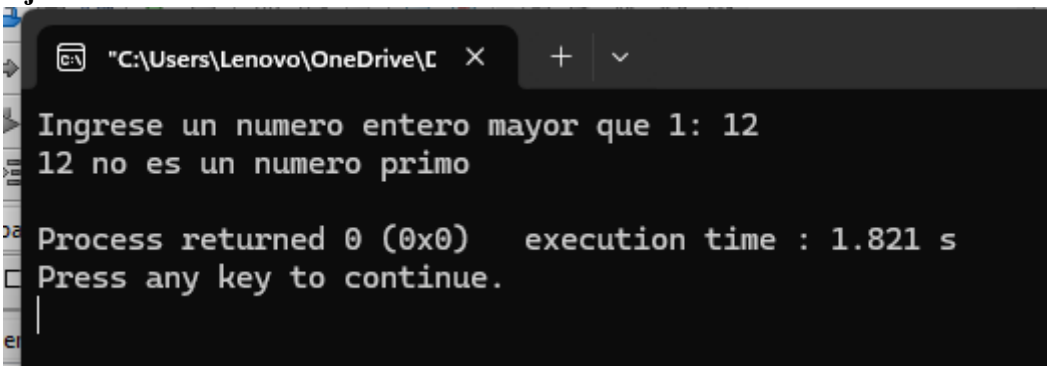
    printf("Ingrese un numero entero mayor que 1: ");
    scanf("%d", &n);

    if (n <= 1) {
        printf("El numero debe ser mayor que 1\n");
    } else {
        while (i < n && es_primo == 1) {
            if (n % i == 0) {
                es_primo = 0;
            } else {
                i++;
            }
        }

        if (es_primo == 1) {
            printf("%d es un numero primo\n", n);
        } else {
            printf("%d no es un numero primo\n", n);
        }
    }

    return 0;
}
```

8.7.Ejecución



9. Números primos comprendidos entre dos números dados

Desarrolla un programa que muestre por pantalla todos los números primos comprendidos entre dos números enteros introducidos por teclado

9.1.Análisis del problema

Entradas: dos números enteros a y b que determinan el intervalo o rango de búsqueda.

Proceso: determinar cuál de los números es menor (inicio) y cual es el mayor (fin)

- si el valor inicio < 2, se debe ajustar automáticamente a 2
- mediante un bucle se recorre cada numero (n) dentro del intervalo
- para cada número, se verifica si es primo comprobando si tiene divisores i desde 2hasta (n-1)
- si no tiene divisores, se imprime en la pantalla y se trabajo con los que si lo tiene

Salidas: los números primos están, dentro de un rango, si no esta dentro se notificara

Restricciones: numero <= 1 no son primos

9.2.Trabal de objetos

Objeto	Nombre	Valor	Tipo
Datos entrada (primer limite)	x	Var	Entero
Dato entrada	Y	Var	Entero

(segundo limite)			
Limite inferior Ordenado y ajustado	Inicio	Var	Entero
Límite superior Ordenado	Fin	Var	Entero
Numero actual elevado en el rango	n	Var	Entero
Divisor de prueba	I	Var	Entero
Indicador de primo (si, no)	Es_primo	Var	Entero
Contador de primos	Encontrados	Var	Entero
Limitite mínimo	2	Cte	Entero

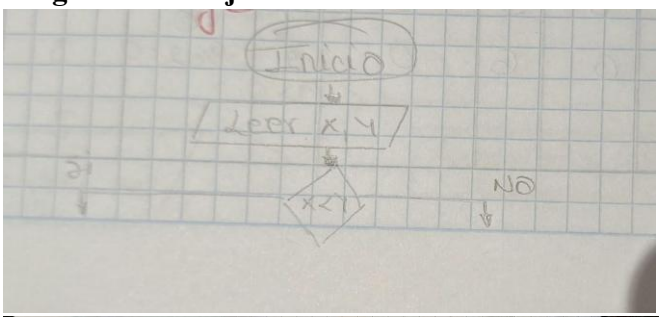
9.3.Seudocódigo

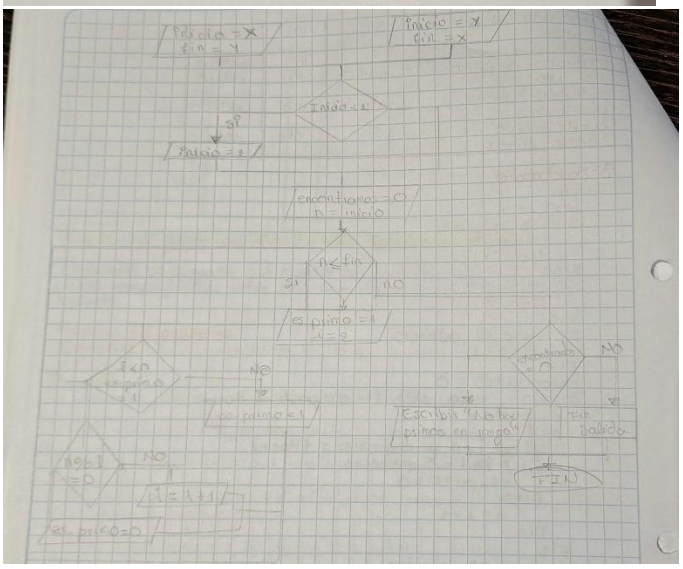
Inicio
Leer x, y
Si a<b entonces
Inicio = x
Fin = y
Sino
Inicio = y
Fin = x
Fin
Si inicio < 2 entonces
Inicio =2
Fin
Encontramos =0
Para n= inicio hasta fin con paso 1 hacer
Es_primo = ¡
1=2
Mientras i<n y es_prino = 1 hacer
Si n% i =0 entonces
Es_primo=0
Sino
I=i+1
Fin
Fin
Si encontramos = 0 entonces
Escribir n
Encontramos = encontramos + 1
Fin
Fin
Si encontramos =0 entonces
Escribir “no existen números primos en el rango”

Fin

Fin

9.4.Diagrama de flujo





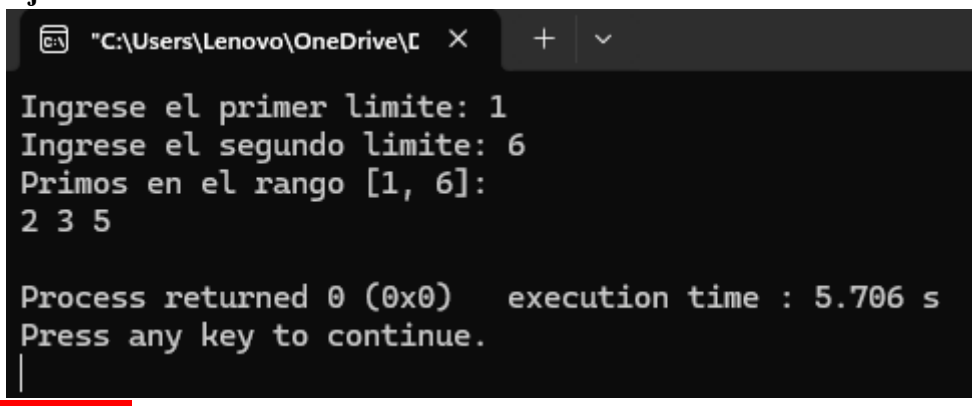
9.5.Prueba de escritorio

x	y	Inicio	fin	n	Es_primo	i	N%i	encontrados	salida
1	6	1→2	6					0	Inicio a 2
1	6	2	6	2	1	2	-(i<n)F	1	Imprime 2
1	6	2	6	3	1	2	3%2=1	2	Imprime 3
1	6	2	6	4	0	2	4%2=0	2	Es_primo =0 (no imprime)
1	6	2	6	5	1	2,3, 4	Ninguno es 0	3	Imprime 5
1	6	2	6	6	0	2	6%2=0	3	Es_primo= 0 (no imprime)
fin								3	Salida 2,3,5

9.6.Código

```c
#include <stdio.h>
int main() {
    int a, b, inicio, fin, num, i, es_primo, encontrados = 0;
    printf("Ingrese el primer limite: ");
    scanf("%d", &a);
    printf("Ingrese el segundo limite: ");
    scanf("%d", &b);
    if (a < b) {
        inicio = a;
        fin = b;
    } else {
        inicio = b;
        fin = a;
    }
    if (inicio < 2) {
        inicio = 2;
    }
    printf("Primos en el rango [%d, %d]:\n", a, b);
    for (num = inicio; num <= fin; num++) {
        es_primo = 1;
        i = 2;
        while (i < num && es_primo == 1) {
            if (num % i == 0) {
                es_primo = 0;
            } else {
                i++;
            }
        }
        if (es_primo == 1) {
            printf("%d ", num);
            encontrados++;
        }
    }
    if (encontrados == 0) {
        printf("No existen numeros primos en este rango.");
    }
    printf("\n");
```

9.7.Ejecución



10. Factorial

Desarrolla un programa que lea un numero entero n=>0 y calcules n =2…….n, sabiendo que 0!=1

10.1. Análisis del problema

Entradas: un numero entero n=>0 ingresado

Proceso: variables acumuladoras multiplicativa en 1 (factorial =1), mediante un bucle (for), se asigna una variable i desde 1 hasta n

- En cada interacción se actualiza el acumulador, multiplicando su valor por i
- Si n=0 no cumple, la variable mantiene su valor inicial 0! =1

Salida: el valor factorial n! calculado

Restricción: n debe ser un entero mayor o igual a cero

10.2. Tabla de objetos

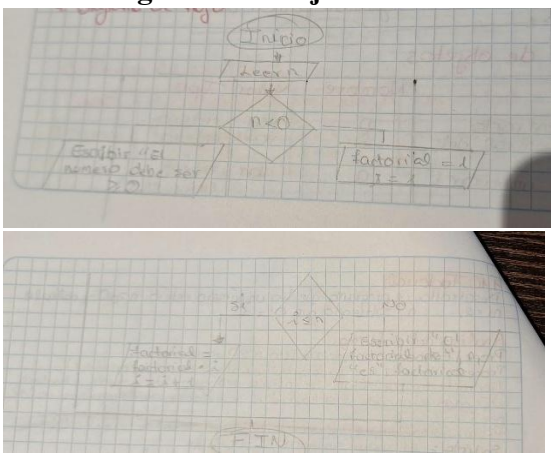
Objeto	Nombre	Valor	Tipo
Dato de entrada	N	Var	Entero
Acumulador del producto factorial	Factorial	Var	Entero grande
Contador interacciones	i	Var	Entero

valor de inicialización	1	Cte	Entero
Limite de restricción	0	Cte	Entero

10.3. Seudocódigo

Inicio
Leer n
Si n<0 entonces
Escribir “el numero debe ser entero no negativo (n>0)”
Sino
Factorial=1
Para i=1 hasta n con paso 1 hacer
Factorial = factorial *i
Fin
Escribir “el factorial de”, n, “es”, factorial
Fin
Fin

10.4. Diagrama de flujo



10.5. Prueba de escritorio

Caso	n	N<0	I	I<=n	factorial	salida
Inicio	5	No	1	Si (1<5)	1*1= 1	Entrada en el bucle
1	5	No	2	Si (2<5)	1*2=2	Continua
2	5	No	3	Si (3<5)	2*3=6	Continua
3	5	No	4	Si (4<5)	6*4=24	Continua
4	5	No	5	Si (5<5)	24*5=120	Continua
5	5	No	6	Si (6<5)	120	Sale del bucle
Fin	5	No			120	La factorial de 5 es 120

10.6. Código

```c
#include <stdio.h>

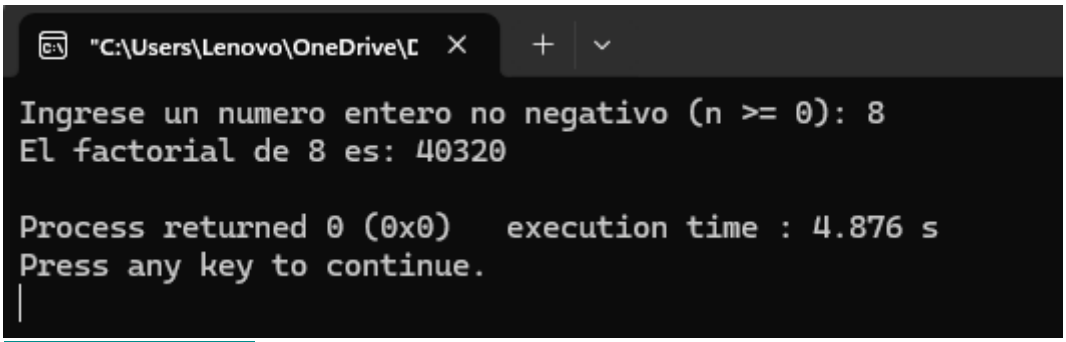
int main() {
    int n, i;
    long long factorial = 1;

    printf("Ingrese un numero entero no negativo (n >= 0): ");
    scanf("%d", &n);

    if (n < 0) {
        printf("Error: El numero debe ser mayor o igual a cero.\n");
    } else {
        for (i = 1; i <= n; i++) {
            factorial *= i;
        }
        printf("El factorial de %d es: %lld\n", n, factorial);
    }

    return 0;
}
```

10.7. Ejecución



11. Serie de Fibonacci

11.1. Análisis del problema

Entradas: Un entero n \ 1 que especifica el número de términos a generar.

Proceso: Generar de forma iterativa los términos con a = 0, b = 1. En cada paso se imprime a, se calcula c = a + b y se resignan a = b y b = c.

Salida: Muestra por pantalla los primeros n términos de Fibonacci.

Restricciones: El valor ingresado para n debe ser mayor o igual a 1 (n \ 1).

11.2. Tabla de objetos

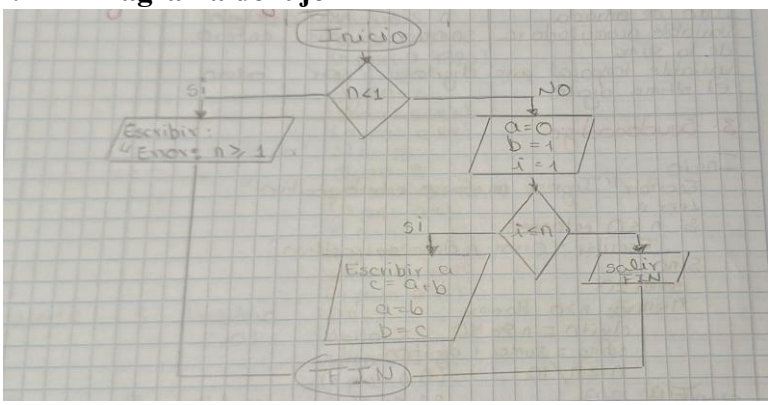
Objeto	Nombre	Valor	Tipo
Dato de entrada	n	Var	Entero
Control del ciclo	i	Var	Entero
Termino anterior	a	Var	Entero (inicia en 0)
Termino actual	b	Var	Entero (inicia en 1)
Siguiente termino	c	Var	Entero

11.3. Seudocódigo

Inicio
Escribir “ingrese un número entero n>=1 :”
Leer n
Si n<1 entonces
Escribir “error n debe ser mayor o igual a 1”
Sino
A=0
B=1

Para i=1 hasta n hacer
Escribir a
C= a+b
A=b
B=c
Fin
Fin
Fin

11.4. Diagrama de lujo



11.5. Prueba de escritorio

Caso	n	validacion	Iteración (i)	Valores (a,b,c)	salida
Normal	7	Verdadero(7>=1)	I=1 I=7	I=1, a=0, c=1, (luego a=1, b=1) i=2, a=1,b=1, c=2 (luego a=1, b=2)	0112
Limite	1	Verdadero(1>=1)	I=1	I=1, a=0, b=1	0
error	0	Falso (0<1)			Error n>=1

11.6. Código

```c
#include <stdio.h>

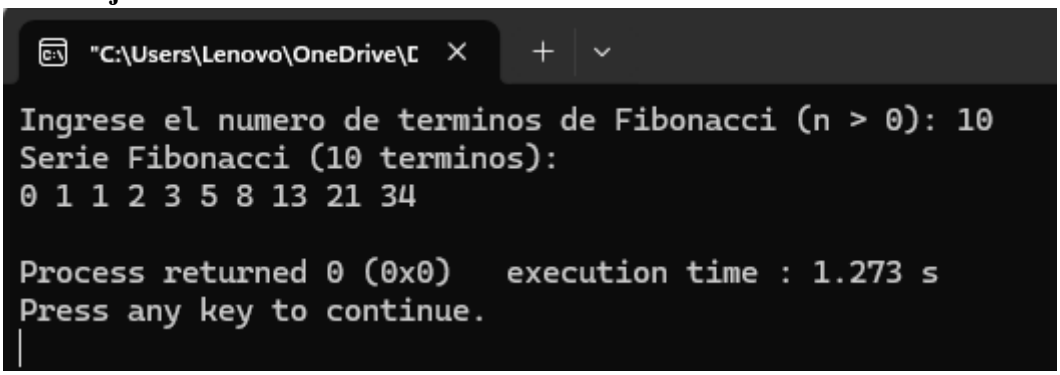
int main() {
    int n, i;
    long long t1 = 0, t2 = 1, siguiente;

    printf("Ingrese el numero de terminos de Fibonacci (n > 0): ");
    scanf("%d", &n);

    if (n <= 0) {
        printf("Ingrese un entero positivo mayor a cero.\n");
    } else {
        printf("Serie Fibonacci (%d terminos):\n", n);
        for (i = 1; i <= n; i++) {
            printf("%lld ", t1);
            siguiente = t1 + t2;
            t1 = t2;
            t2 = siguiente;
        }
        printf("\n");
    }

    return 0;
}
```

11.7. Ejecución



12. Suma de dígitos

Desarrolla un programa que lea en entero positivo y calcule la suma de sus digitos

12.1. Análisis del problema

Entradas: numero entero positivo
Proceso: calcular la suma de los dígitos que componen el numero
Salida: mostrar el resultado de la suma de los dígitos
Restricciones: el numero ingresado debe ser estrictamente positivo

12.2. Tabla de objetos

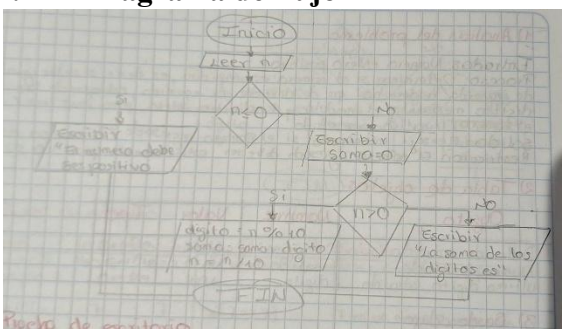
Objeto	Nombre	valor	tipo
Dato de entrada	n	Var	Entero
Variable acumuladora de la suma	Suma	Var	Entero
Variable temporal para el ultimo digito	Digito	Var	Entero

12.3. Seudocódigo

Inicio
Escribir “ingrese un número entero positivo”
Leer n
Si n<=0 entonces

Escribir “error n debe ser positivo”
Sino
Suma =0
Mientras n>0 hacer
Digito =n%10
Suma = suma + digito
N= n/10
Fin
Escribir “ la suma de los dígitos es:”, suma
Fin
Fin

12.4. Diagrama de flujo



12.5. Prueba de escritorio

Caso	**n**	**validación**	**Iteraciones del código**	**Salida**
Normal	4825	Verdadero (4825>0)	N=4825, dígitos =5 (suma =0 +5=5) (n pasa a 4825)	19
Limite	5	Verdadero (5>0)	N=5, digito =5 suma =0+5=5 (n pasa a 0)	5
Error	0	Falso (0<=0)	0 no se ejecuta	Error “el numero debe ser positivo”

12.6. Código

```c
#include <stdio.h>

int main() {
    long long n, aux;
    int suma = 0;
    printf("Ingrese un numero entero: ");
    scanf("%lld", &n);

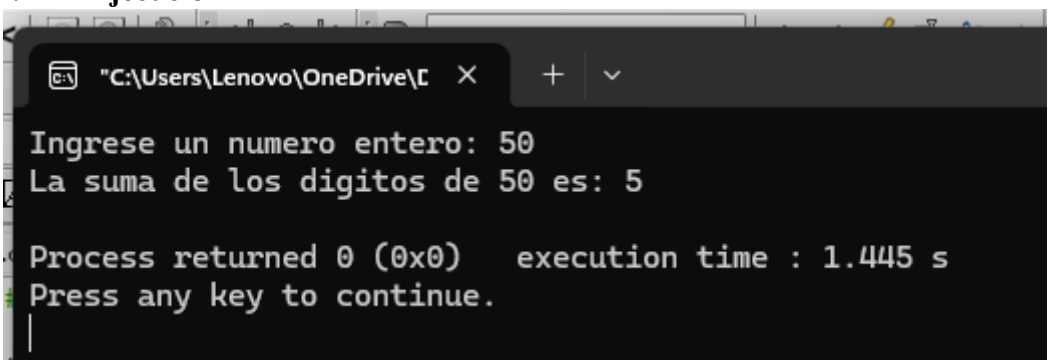
    aux = (n < 0) ? -n : n;

    while (aux > 0) {
        suma += (int)(aux % 10);
        aux /= 10;
    }

    printf("La suma de los digitos de %lld es: %d\n", n, suma);

    return 0;
}
```

12.7. Ejecución



13. Numero capicúa

Desarrolla un programa que lea un entero positivo y determine si es capicúa, es decir, si se lee igual de izquierda a derecha que de derecha a izquierda

13.1. Análisis del problema

Entradas: numero entero positivo

Proceso: determinar si el numero es capicúa, es decir, si se lee igual de izquierda a derecha que de derecha a izquierda, extrayendo los dígitos mediante el módulo y la división entera para construir el numero invertido acumulado

Salida: mensaje indicando si el numero ingresado es capicúa o no

Restricciones: el numero ingresado debe ser estrictamente positivo n>0

13.2. Tabla de objetos

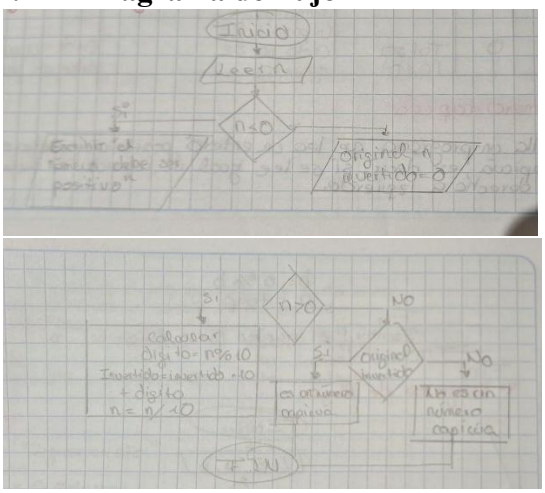
objeto	nombre	valor	tipo
Dato de entrada	n	var	entero
Numero invertido	invertido	Var	entero
Ultimo digito extraido	digito	Var	entero

13.3. Seudocódigo

Inicio
Escribir “ingrese un numero entero positivo”
Leer n
Si <=0 entonces
Escribir “error : el numero debe ser positivo”
Sino
Original = n
Invertido =0

Mientras n>0 hacer
Digito =n%10
Invertido = invertido *10+digito
N=n/10
Fin
Si original = invertido entonces
Escribir original “es un numero capicúa”
Sino
Escribir original “no es un numero capicúa”
Fin
fin

13.4. Diagrama de flujo



13.5. Prueba de escritorio

Caso	N inicial original	Original= invertido	salida
Normal	**12321**	**Si (12321==12321)**	**12321 es número capicúa**
Limite	**7**	**Si(7==7)**	**7 es un numero capicúa**
Termina 0	**120**	**No (120!=21)**	**120 no es un numero capicúa**

13.6. Código

```c
#include <stdio.h>

int main() {
    long long num, aux, invertido = 0;

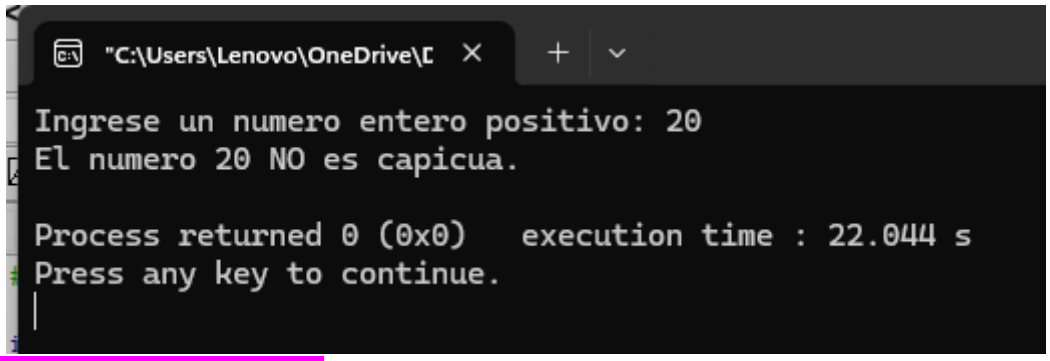
    printf("Ingrese un numero entero positivo: ");
    scanf("%lld", &num);

    if (num < 0) {
        printf("El numero debe ser no negativo.\n");
    } else {
        aux = num;
        while (aux > 0) {
            invertido = (invertido * 10) + (aux % 10);
            aux /= 10;
        }

        if (num == invertido) {
            printf("El numero %lld ES capicua.\n", num);
        } else {
            printf("El numero %lld NO es capicua.\n", num);
        }
    }

    return 0;
}
```

13.7. Ejecución



14. Máximo común divisor

Desarrolla un programa que lea dos enteros positivos a y b y calcule su máximo común divisor con el algoritmo de Eulisos mientras, b diferente de 0 reemplaza (a,b) por (b,a%b). al terminar a es MCD

14.1. Análisis del problema

Entradas: dos enteros positivos a y b

Proceso: calcular MCD utilizando el algoritmo de Euclides mientras b diferente de 0 se reemplaza (a,b) (b,a% b). al terminar a es el MCD.

Salida: mostrar el valor del máximo común divisor

Restricciones: los números ingresados a y b deben ser estrictamente positivos a>0 y b>0

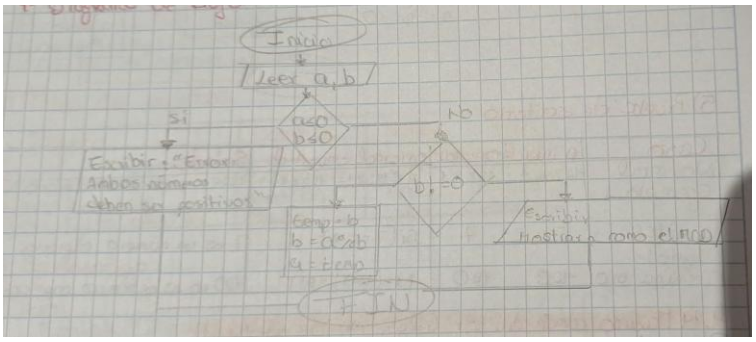
14.2. Tabla de objetos

objeto	nombre	Valor	tipo
Dato de entrada 1	A	var	Entero
Dato de entrada 2	B	Var	Entero
Variable temporal auxiliar	temp	Var	Entero

14.3. Seudocódigo

Inicio
Escribir “ingrese dos enteros positivos a y b”
Leer a, b
Si a<=0 o b<=0 entonces
Escribir “error ambos números deben ser positivos”
Sino
Mientras b!=0 hacer
Temp=b
B=a%b
A= temp
Fin
Escribir “el máximo común divisor es:”, a
Fin
fin

14.4. Diagrama de flujo



14.5. Prueba de escritorio

caso	A inicial	B inicial	Recorrido de bucle	Final MCD	salida
Normal	48	18	A=48, b=18, (48%18=12)	6	El máximo común divisor
Limite	10	5	A=10, b=5 (10%5=0) A=5 b=0	5	El máximo común divisor
Error	-5	10	Validacion a<=0 se cumple de inmediato		Error ambos números deben ser positivos

14.6. Código

```c
#include <stdio.h>

int main() {
    int a, b, temp, num1, num2;

    printf("Ingrese el primer entero positivo: ");
    scanf("%d", &num1);
    printf("Ingrese el segundo entero positivo: ");
    scanf("%d", &num2);

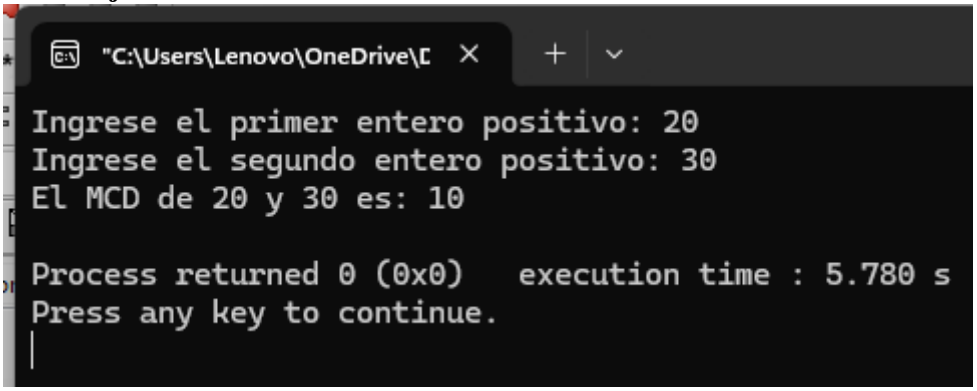
    if (num1 <= 0 || num2 <= 0) {
        printf("Ambos numeros deben ser enteros positivos.\n");
    } else {
        a = num1;
        b = num2;

        while (b != 0) {
            temp = b;
            b = a % b;
            a = temp;
        }

        printf("El MCD de %d y %d es: %d\n", num1, num2, a);
    }

    return 0;
```

14.7. Ejecución



15. Año bisiesto

Desarrolla un programa que lea un año y determine. Un año es bisiesto si es divisible entre 4, excepte los años divisibles entre 100, que solo lo son si también son divisibles entre 400

15.1. Análisis del problema

Entradas: numero entero que representa el año ingresado
Proceso: un año bisiesto si es divisable entre 4, excepto los años divisibles entre 100, que solo lo son si también son divisibles entre 400.
Salida: mensaje indicando si el año ingresado es bisiesto o no
Restricciones: el año debe ser un numero entero positivo

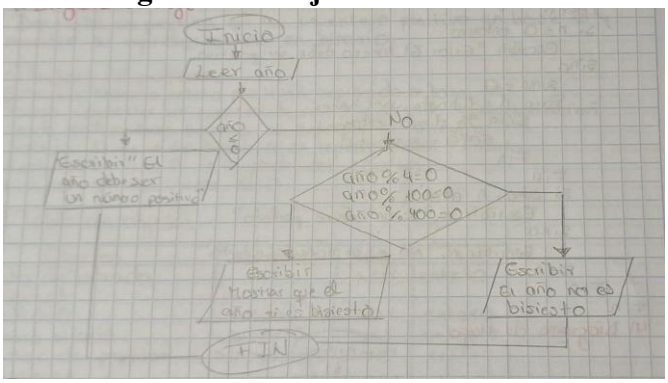
15.2. Tabla de objetos

objeto	nombre	Valor	Tipo
Datos de entrada	Año	Var	Entero
Constantes de división	4, 100,400	var	entero

15.3. Seudocódigo

Inicio
Escribir “ingrese un año”
Leer año
Si año <=0 entonces
Escribir “error el año debe ser un numero positivo”
Sino
Si (año%4=0y año %100=0) o (año%400=0) entonces
Escribir año “si es un año bisiesto”
Sino
Escribir año “no es un año bisiesto”
Fin
Fin
fin

15.4. Diagrama de flujo



15.5. Prueba de escritorio

Caso	Año	validación	salida
Normal	2024	Verdadero	2024 si es un año bisiesto
normal	1900	Verdadero	1900 no es un año bisiesto
No bisiesto genérico	2023	verdadero	2023 no es un año bisiesto

Error	-2024	Falso	Error el año debe ser un numero entero

15.6. Código

```c
#include <stdio.h>

int main() {
    int anio;

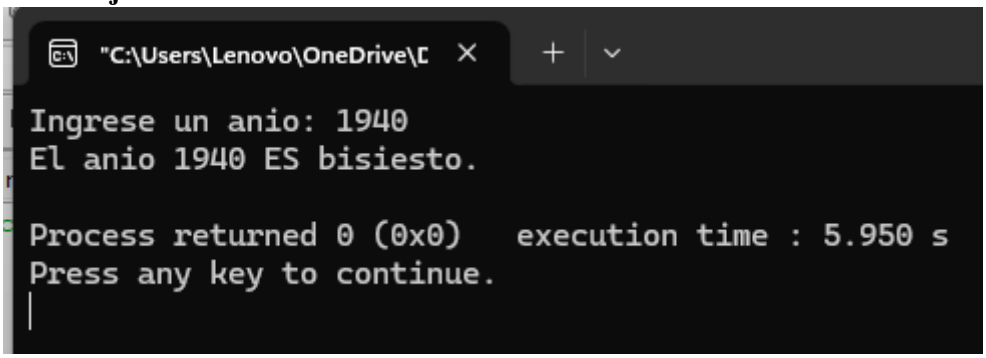

    printf("Ingrese un anio: ");
    scanf("%d", &anio);


    if (anio <= 0) {
        printf("Error: Ingrese un anio valido mayor que cero.\n");
    } else {

        if ((anio % 4 == 0 && anio % 100 != 0) || (anio % 400 == 0)) {
            printf("El anio %d ES bisiesto.\n", anio);
        } else {
            printf("El anio %d NO es bisiesto.\n", anio);
        }
    }

    return 0;
}
```

15.7. Ejecución



16. Numero perfecto

Desarrolla un programa que lea un entero positivo y determine si es perfecto, es decir, si es igual a la suma de sus divisores propios.

16.1. Análisis del problema

Entrada: Número entero positivo ingresado por teclado
Proceso: Determinar si un número es perfecto, es decir, si es igual a la suma de sus divisores propios
Salida: Mensaje en la pantalla indicando si el número ingresado es perfecto
Restricción: El número ingresado debe ser estrictamente positivo (n>0)

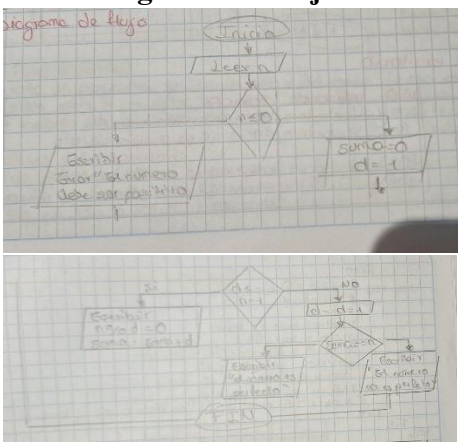
16.2. Tabla de objetos

Objeto	Nombre	Valor	Tipo
Dato de entrada	n	var	entero
Variable de control	d	var	entero
Variable acumuladora de la suma	suma	var	entero

16.3. Seudocódigo

```
Inicio
  Escribir "Ingrese un número entero positivo:"
  Leer n
  si n <= 0 entonces
    Escribir "Error: El número debe ser positivo"
  sino
    suma = 0
    Para d = 1 hasta n - 1 hacer
      si n % d == 0 entonces
        suma = suma + d
      Fin
    Fin
    si suma = n entonces
      Escribir n, " es un número perfecto"
    sino
      Escribir n, " no es un número perfecto"
    Fin
  Fin
Fin
```

16.4. Diagrama de flujo



16.5. Prueba de escritorio

caso	n	Validación	d	y n % d = 0	suma	suma = n	Salida
Normal Ejemplo	28	Verdadero	1, 2, 4, 7, 14		28	Si (28=28)	28 es perfecto
Normal No perfecto	10	Verdadero	1, 2, 5		8	No (8 = 10)	10 no es número perfecto
Límite	1	Verdadero	1 hasta 0		0	No (0 = 1)	No perfecto

16.6. Código

```c
#include <stdio.h>

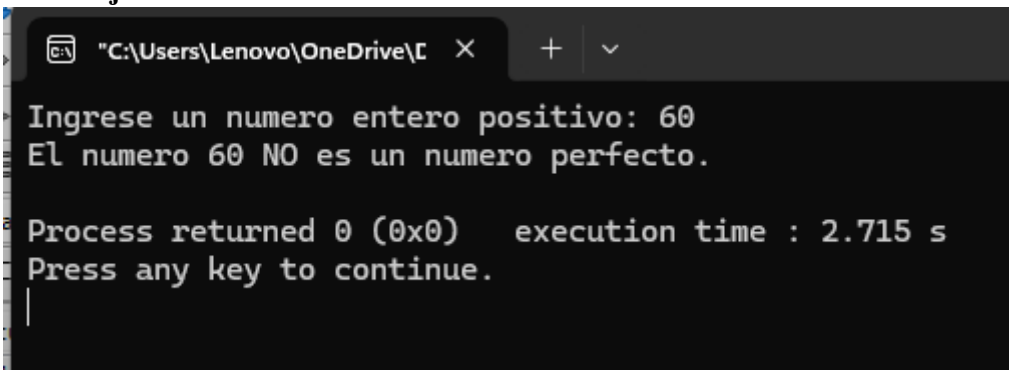
int main() {
    int num, i, suma = 0;

    printf("Ingrese un numero entero positivo: ");
    scanf("%d", &num);

    if (num <= 0) {
        printf("Error: Ingrese un numero entero mayor que cero.\n");
    } else {
        for (i = 1; i <= num / 2; i++) {
            if (num % i == 0) {
                suma += i;
            }
        }
        if (suma == num) {
            printf("El numero %d ES un numero perfecto.\n", num);
        } else {
            printf("El numero %d NO es un numero perfecto.\n", num);
        }
    }

    return 0;
}
```

16.7. Ejecución



17. Conversión de decimal a binario

Desarrolla un programa que lea un entero positivo en base 10 y muestre su representación en binario.

17.1. Análisis del problema

Entradas: Número entero positivo expresado en base 10

Proceso: Convertir un número decimal a binario en programación usando matemáticas básicas sin depender de arreglos.

Salida: Número traducido a su representación binaria

Restricción: n >= 0 -> no puede ser negativo. Si n = 0 el resultado es 0.

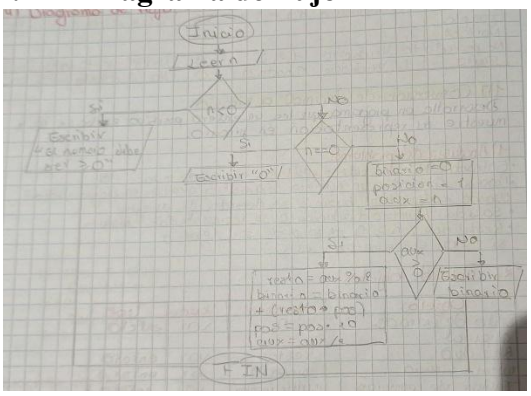
17.2. Tabla de objetos

Objeto	Nombre	Valor	Tipo
Dato de entrada	n	var	entero
Original			
Residuo	resto	var	entero
Número binario	binario	var	entero
Factor de posición decimal	multiplicador	var	entero

17.3. Seudocódigo

```
Inicio
  Leer n
  si n < 0 entonces
    Escribir "El número debe ser un entero no negativo (n>=0)"
  sino
    si n = 0 entonces
      Escribir "El número ", n, " en binario es: 0"
    sino
      binario = 0
      posicion = 1
      aux = n
      mientras aux > 0 hacer
        resto = aux % 2
        binario = binario + (resto * posicion)
        posicion = posicion * 10
        aux = aux / 2
      Fin
      Escribir "El número ", n, " en binario es: ", binario
    Fin
  Fin
Fin
```

17.4. Diagrama de flujo



17.5. Prueba de escritorio

Caso	n	Recorrido del ciclo	Salida
Normal Ejemplo	13	1. n=13, resto=1, binario=0+(1*1)=1, mult=10, n=6 2. n=6, resto=0, binario=1+(0*10)=1, mult=100, n=3 3. n=3, resto=1, binario=1+(1*100)=101, mult=1000, n=1 4. n=1, resto=1, binario=101+(1*1000)=1101, mult=10000, n=0	El número 13 en binario es: 1101

17.6. Código

```c
#include <stdio.h>

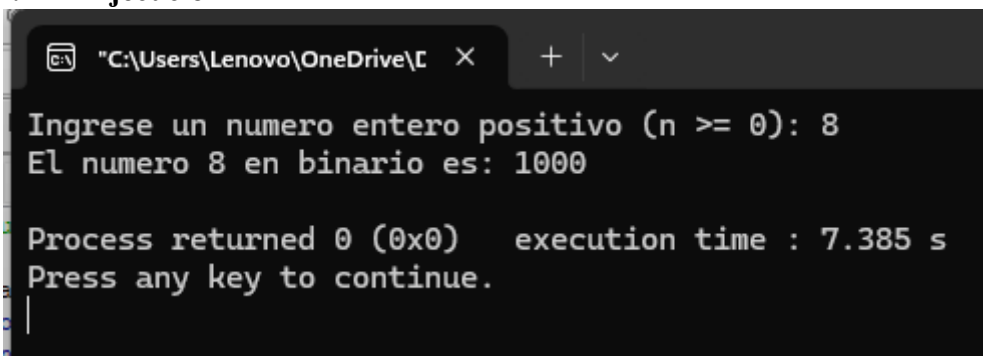
int main() {
    long long n, aux, binario = 0, posicion = 1;
    int resto;

    printf("Ingrese un numero entero positivo (n >= 0): ");
    scanf("%lld", &n);

    if (n < 0) {
        printf("El numero debe ser un entero no negativo (n >= 0).\n");
    } else if (n == 0) {
        printf("El numero 0 en binario es: 0\n");
    } else {
        aux = n;
        while (aux > 0) {
            resto = aux % 2;
            binario = binario + (resto * posicion);
            posicion = posicion * 10;
            aux = aux / 2;
        }
        printf("El numero %lld en binario es: %lld\n", n, binario);
    }

    return 0;
}
```

17.7. Ejecución



18. Conversión de binario a decimal

Desarrolla un programa que lea un entero n >= 1 y dibuje por pantalla un triángulo rectángulo de n filas i tiene i asteriscos.

18.1. Análisis del problema

Entradas: Un número entero binario

Salida: El valor equivalente traducido al sistema decimal

Restricción: El número no debe ser negativo, utilizar solo 1 o 0

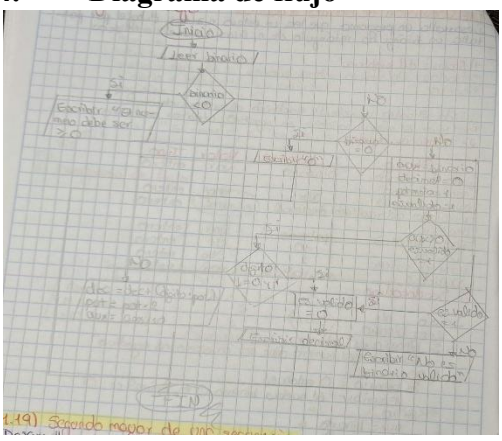
18.2. Tabla de objetos

Objeto	Nombre	Valor	Tipo
Dato de entrada (número binario)	binario	var	entero
auxiliar binario	aux	var	entero
acumulador del resultado decimal	decimal	var	entero
Factor potencia	potencia	var	entero
Divisor	10	cte	entero
Base de sistema	2	cte	entero
validez	es_valido	var	entero

18.3. Seudocódigo

```
Inicio
  Leer binario
  Si binario < 0 entonces
    Escribir "El número debe ser un entero no negativo"
  Sino
    Si binario = 0 entonces
      Escribir "El número binario 0 en decimal es: 0"
    Sino
      aux = binario
      decimal = 0
      potencia = 1
      es_valido = 1
      Mientras aux > 0 y es_valido = 1 Hacer
        digito = aux % 10
        Si digito != 0 y digito != 1 entonces
          es_valido = 0
        Sino
          decimal = decimal + (digito * potencia)
          potencia = potencia * 2
          aux = aux / 10
        Fin
      Fin
      Si es_valido = 1 Entonces
        Escribir "El número binario ", binario, " es decimal es: ", decimal
      Sino
        Escribir "El número no es un binario valido"
      Fin
    Fin
  Fin
Fin
```

18.4. Diagrama de flujo



18.5. Código

```c
#include <stdio.h>

int main() {
    long long binario, aux, decimal = 0, potencia = 1;
    int digito, es_valido = 1;

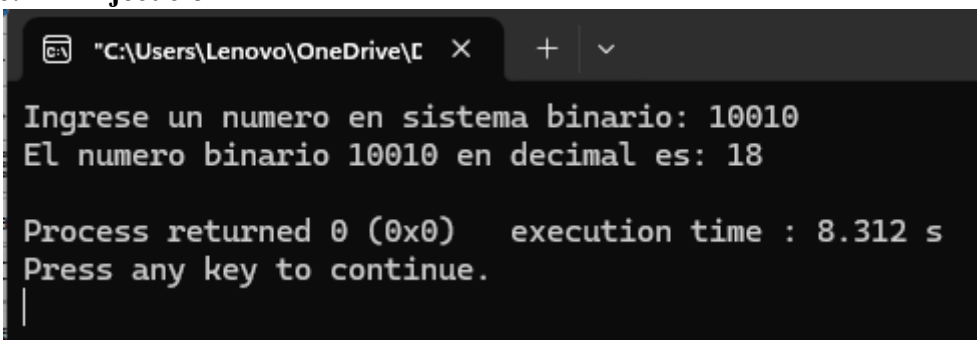
    printf("Ingrese un numero en sistema binario: ");
    scanf("%lld", &binario);

    if (binario < 0) {
        printf("El numero debe ser no negativo.\n");
    } else if (binario == 0) {
        printf("El numero binario 0 en decimal es: 0\n");
    } else {
        aux = binario;
        while (aux > 0 && es_valido == 1) {
            digito = aux % 10;
            if (digito != 0 && digito != 1) {
                es_valido = 0;
            } else {
                decimal = decimal + (digito * potencia);
                potencia = potencia * 2;
                aux = aux / 10;
            }
        }

        if (es_valido == 1) {
            printf("El numero binario %lld en decimal es: %lld\n", binario, decimal);
        } else {
            printf("Error: El numero ingresado contiene digitos distintos de 0 y 1.\n");
        }
    }

    return 0;
}
```

18.6. Ejecución



19. Segundo mayor de una secuencia

Desarrolla un programa que lea números enteros positivos hasta que se introduzcan un 0, y muestre el mayor y el segundo mayor de los valores ingresados. Si hay menos de dos valores, debe informarlo

19.1. Análisis del problema

Entradas: Una secuencia indeterminada de números enteros positivos finalizada por el valor centinela 0

Proceso: El algoritmo leerá números enteros uno a uno hasta que se ingresa un 0, utilizando un contador para registrar los valores y dos variables inicializadas en 0 (mayor y segundo) para identificar los dos valores más grandes. Para cada número positivo, si este supera al mayor, el antiguo máximo pasa a ser el segundo y el nuevo toma el puesto principal

Salida: Muestra valores de mayor y segundo si el contador es menor a 2, se muestra un mensaje advirtiendo que faltan datos para determinar el segundo mayor.

Restricciones: Solo se toman en cuenta números estrictamente positivos (num>0). El centinela 0 solo sirve para tomar lectura.

19.2. Tabla de objetos

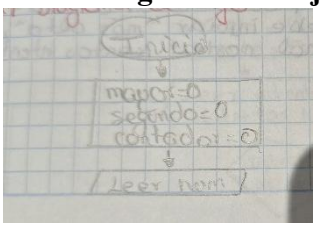
Objetos	Nombre	Valor	Tipo
Dato de entrada (número leído)	num	var	entero
Mayor valor	mayor	var	entero

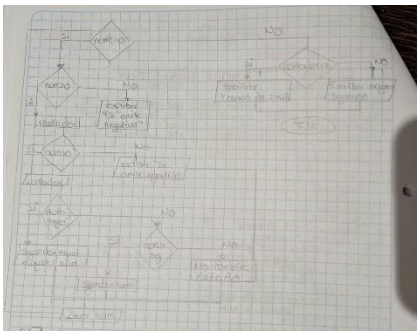
Segundo mayor valor	segundo	var	entero
Contador de valores	contador	var	entero
Centinela	0	cte	entero

19.3. Seudocódigo

```
Inicio
  mayor = 0
  segundo = 0
  contador = 0
  Leer num
  Mientras num != 0 Hacer
    Si num > 0 entonces
      contador = contador + 1
      Si num > mayor entonces
        segundo = mayor
        mayor = num
      Sino
        Si num > segundo entonces
          segundo = num
        Fin
      Fin
    Sino
      Escribir "Ingrese únicamente enteros positivos (se omite negativo)"
    Fin
    Leer num
  Fin
  Si contador < 2 entonces
    Escribir "Se ingresaron menos de dos valores validos"
  Sino
    Escribir "Mayor = ", mayor
    Escribir "Segundo mayor = ", segundo
  Fin
Fin
```

19.4. Diagrama de flujo

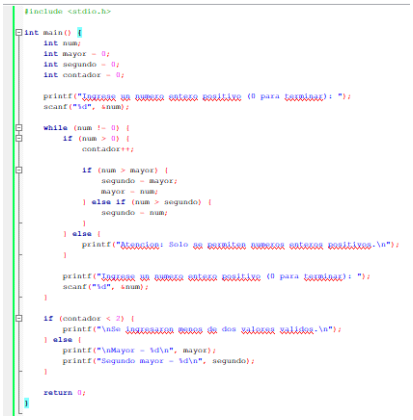




19.5. Prueba de escritorio

Iteración	num	num != 0	contador
Inicio	7	Si	0 -> 1
1	3	Si	1 -> 2
2	9	Si	2 -> 3
3	5	Si	3 -> 4
4	0	NO	4
Fin			4 >= 2

19.6. Código



```c
#include <stdio.h>

int main() {
    int num;
    int mayor = 0;
    int segundo = 0;
    int contador = 0;

    printf("Ingrese un numero entero positivo (0 para terminar): ");
    scanf("%d", &num);

    while (num != 0) {
        if (num > 0) {
            contador++;

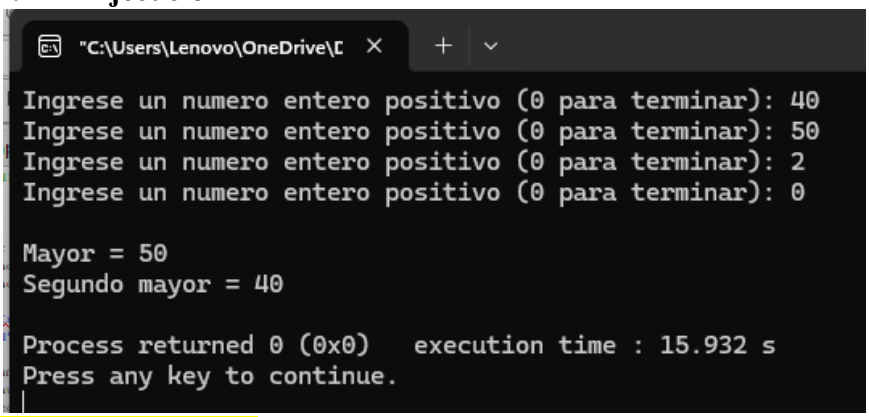
            if (num > mayor) {
                segundo = mayor;
                mayor = num;
            } else if (num > segundo) {
                segundo = num;
            }
        } else {
            printf("Atencion: Solo se permiten numeros enteros positivos.\n");
        }

        printf("Ingrese un numero entero positivo (0 para terminar): ");
        scanf("%d", &num);
    }

    if (contador < 2) {
        printf("\nSe ingresaron menos de dos valores validos.\n");
    } else {
        printf("\nMayor = %d\n", mayor);
        printf("Segundo mayor = %d\n", segundo);
    }

    return 0;
}
```

19.7. Ejecución



```
"C:\Users\Lenovo\OneDrive\[
Ingrese un numero entero positivo (0 para terminar): 40
Ingrese un numero entero positivo (0 para terminar): 50
Ingrese un numero entero positivo (0 para terminar): 2
Ingrese un numero entero positivo (0 para terminar): 0

Mayor = 50
Segundo mayor = 40

Process returned 0 (0x0)   execution time : 15.932 s
Press any key to continue.
```

20. Adivina el numero

Desarrolla un programa que "piense" un número entre 1 y 100 (puede ser fijo para la práctica) y pida al usuario adivinarlo. Después de cada intento debe

iniciar "más alto", "más bajo" o "acertaste" y al final mostrar cuantos intentos se usaron.

20.1. Análisis del problema

Entradas: intentos numéricos enteros ingresados consecutivamente por el usuario.

Proceso: Un juego de adivinanza donde el sistema genera un número secreto entre 1 y 100 y el usuario intenta descubrirlo mediante un bucle el cual incrementa un contador de intentos con cada turno. En cada iteración, el programa evalúa el número ingresado y da una pista al usuario.

Salida: Mensajes guía cada tras intentos.

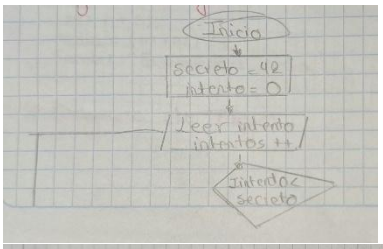
20.2. Tabla de objetos

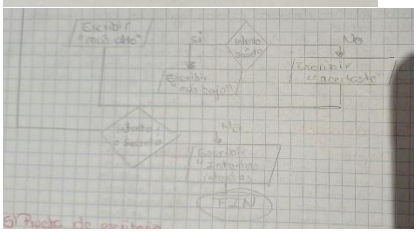
Objeto	Nombre	Valor	Tipo
Número secreto a adivinar	secreto	var/cte	entero
Intento ingresado por el usuario	intento	var	entero
Contador de intentos	intentos	var	entero
Límite inferior del rango	1	cte	entero
Límite superior del rango	100	cte	entero

20.3. Seudocódigo

```
Inicio
  Secreto = 42
  intentos = 0
  Repetir
    Escribir "Ingrese su intento (1 al 100)"
    Leer intento
    Intentos = intentos + 1
    Si intento < secreto entonces
      Escribir "más alto"
    Sino
      Si intento > secreto entonces
        Escribir "más bajo"
      Sino
        Escribir "acertaste"
      Fin
    Fin
  Hasta que intento = secreto
  Escribir "Número total de intentos usados", intentos
Fin
```

20.4. Diagrama de flujo





20.5. Prueba de escritorio

Iteración	intento	intentos	intento < secreto
1	50	1	NO
2	25	2	Si (25 < 42)
3	37	3	Si (37 < 42)
4	43	4	NO
5	40	5	Si (40 < 42)
6	41	6	Si (41 < 42)
7	42	7	NO

20.6. Código

```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int main() {
    int secreto, intento, intentos = 0;

    srand(time(NULL));
    secreto = (rand() % 100) + 1;

    printf("=== JUEGO: ADIVINA EL NUMERO (1 al 100) ===\n\n");

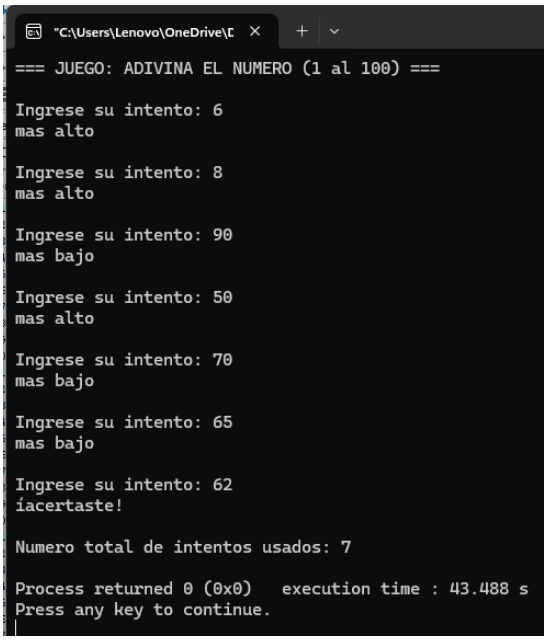
    do {
        printf("Ingrese su intento: ");
        scanf("%d", &intento);
        intentos++;

        if (intento < secreto) {
            printf("mas alto\n\n");
        } else if (intento > secreto) {
            printf("mas bajo\n\n");
        } else {
            printf("¡acertaste!\n\n");
        }
    } while (intento != secreto);

    printf("Numero total de intentos usados: %d\n", intentos);

    return 0;
```

20.7. Ejecución


"C:\Users\Lenovo\OneDrive\[
=== JUEGO: ADIVINA EL NUMERO (1 al 100) ===
Ingrese su intento: 6
mas alto
Ingrese su intento: 8
mas alto
Ingrese su intento: 90
mas bajo
Ingrese su intento: 50
mas alto
Ingrese su intento: 70
mas bajo
Ingrese su intento: 65
mas bajo
Ingrese su intento: 62
¡acertaste!
Numero total de intentos usados: 7
Process returned 0 (0x0) execution time : 43.488 s
Press any key to continue.